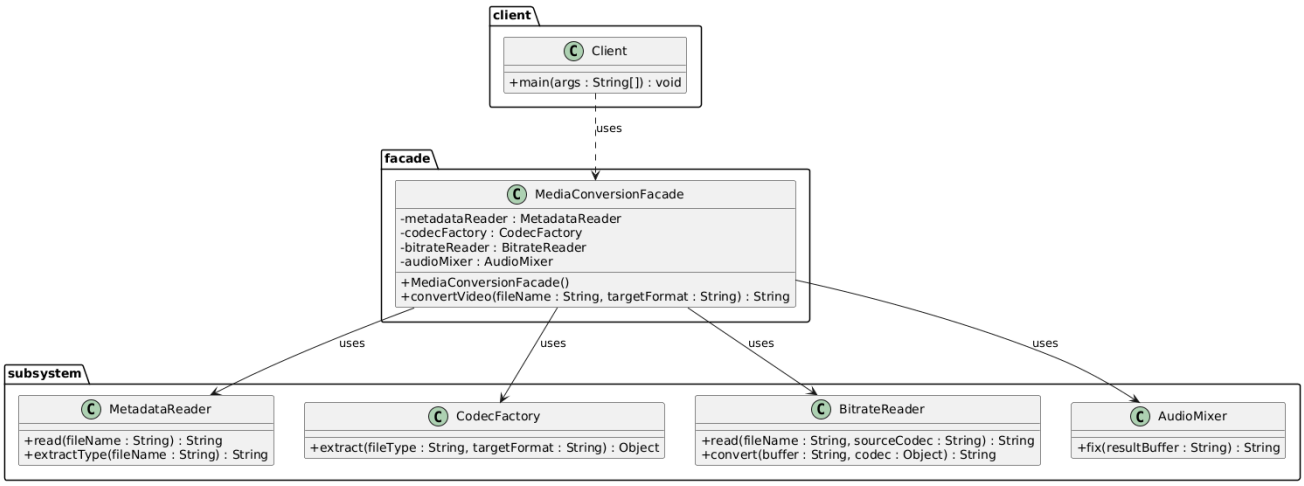
Sanzhar Abayev SE-2538. Assignment 4 — Facade Pattern. Report

1. Introduction

The **Media/File Conversion Facade** addresses the inherent complexity of media conversion frameworks. In a raw multimedia system, converting a file from one format to another requires a client to coordinate multiple distinct, specialized components: extracting file headers via MetadataReader, selecting compatible codecs through CodecFactory, managing stream buffers with BitrateReader, and synchronizing audio/video channels via AudioMixer.

Directly exposing these classes to high-level clients creates tight coupling, high maintenance overhead, and steep learning curves. The Facade pattern introduces a unified interface (MediaConversionFacade) that encapsulates this subsystem, reducing a multi-step, error-prone workflow into a single, clean method call (convertVideo).

2. UML Class Diagram



3. Clean Code Principles & Justification

Principle 1: Meaningful Names

- **Justification:** Classes and methods use descriptive domain terminology (MetadataReader, CodecFactory, convertVideo) that instantly communicates their purpose without requiring deep architectural comments.
- **Excerpt:**

```
// Before: generic naming
public Object proc(String f, String t) { ... }
// After: self-explanatory naming
public String convertVideo(String fileName, String targetFormat) { ... }
```

Principle 2: Single Responsibility Principle (SRP)

- **Justification:** The Facade class is strictly responsible for *coordinating* the workflow sequence. It contains zero business logic for parsing, codec matching, or stream manipulation; those responsibilities are cleanly isolated within their respective subsystem classes.

Principle 3: Encapsulation & Information Hiding

- **Justification:** The client is entirely shielded from subsystem internals. By keeping subsystem classes package-private or hidden behind the facade API, changes to BitrateReader or CodecFactory algorithms will not break client code.

Principle 4: Small, Focused Classes

- **Justification:** Instead of creating a monolithic "God Class" that handles all processing steps, the subsystem is broken into small, cohesive classes (MetadataReader, BitrateReader, etc.), each handling a single well-defined aspect of conversion.

Principle 5: Robust Exception / Flow Handling

- **Justification:** The facade organizes execution flow in a predictable, linear sequence. If validation or conversion steps fail, error handling can be centralized cleanly inside the facade tier rather than scattered across multiple UI or client entry points.

4. Conclusion

- **What the Facade Hides:** The Facade completely hides the underlying subsystem instantiation order, complex parameter passing between intermediate codecs, stream buffers, and detailed audio mixing configurations.

- **What Remains Directly Accessible:** In accordance with the structural design pattern, subsystem classes (MetadataReader, CodecFactory, etc.) remain fully public and decoupled. Advanced clients or system tests can bypass the facade and interact with subsystem classes directly if low-level, granular control is required.

5. Link to your GitHub repository

https://github.com/Sanzh-123/assignment-4-facade-pattern-java